

# PlantApp

Red Social de Plantas vs. Zombies

---

## Documentación Técnica — Parcial 2

**Autor:** Andrés Marcelino Escobar

**Profesor:** Adolfo Centeno

**Entrega:** 8 de Mayo de 2026

Flutter | Spring Boot | PostgreSQL | Docker | Render

Principios de Diseño de Software

## Contents

---

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Descripción General del Proyecto</b>                                 | <b>3</b>  |
| 1.1      | Motivación . . . . .                                                    | 3         |
| 1.2      | Estructura General del Proyecto . . . . .                               | 3         |
| <b>2</b> | <b>Tecnologías Utilizadas</b>                                           | <b>3</b>  |
| <b>3</b> | <b>Principios de Diseño Utilizados</b>                                  | <b>4</b>  |
| 3.1      | Principio de Responsabilidad Única (SRP) . . . . .                      | 4         |
| 3.1.1    | Aplicación en el Backend . . . . .                                      | 4         |
| 3.1.2    | Aplicación en el Frontend . . . . .                                     | 5         |
| 3.2      | Principio Abierto/Cerrado (OCP) . . . . .                               | 5         |
| 3.3      | Principio de Inversión de Dependencias (DIP) . . . . .                  | 6         |
| 3.4      | Principio DRY (Don't Repeat Yourself) . . . . .                         | 6         |
| 3.5      | Principio KISS (Keep It Simple, Stupid) . . . . .                       | 7         |
| 3.6      | Principio de Separación de Intereses (Separation of Concerns) . . . . . | 8         |
| <b>4</b> | <b>Patrones de Diseño Utilizados</b>                                    | <b>8</b>  |
| 4.1      | Patrón Repository . . . . .                                             | 8         |
| 4.1.1    | Definición . . . . .                                                    | 8         |
| 4.1.2    | Beneficio principal . . . . .                                           | 8         |
| 4.1.3    | Repositorios implementados . . . . .                                    | 8         |
| 4.2      | Patrón MVC (Model-View-Controller) . . . . .                            | 9         |
| 4.2.1    | Definición . . . . .                                                    | 9         |
| 4.2.2    | Aplicación en dos niveles . . . . .                                     | 9         |
| 4.2.3    | Diagrama de flujo MVC . . . . .                                         | 9         |
| 4.3      | Patrón de Diseño Facade (mencionado como extensión) . . . . .           | 10        |
| <b>5</b> | <b>Modelos / Entidades JPA</b>                                          | <b>10</b> |
| 5.1      | Tweet.java — Entidad principal . . . . .                                | 10        |
| 5.2      | Usuario.java . . . . .                                                  | 11        |
| 5.3      | Reaccion.java . . . . .                                                 | 11        |
| 5.4      | Comentario.java . . . . .                                               | 11        |
| <b>6</b> | <b>Endpoints de la API REST</b>                                         | <b>12</b> |
| <b>7</b> | <b>Arquitectura y Despliegue</b>                                        | <b>12</b> |
| 7.1      | Diagrama de arquitectura . . . . .                                      | 12        |
| 7.2      | Configuración de base de datos . . . . .                                | 13        |
| 7.3      | Dockerfile . . . . .                                                    | 13        |
| 7.4      | Compilación del APK . . . . .                                           | 14        |
| <b>8</b> | <b>Funcionalidades de la Aplicación</b>                                 | <b>14</b> |
| 8.1      | Autenticación . . . . .                                                 | 14        |
| 8.2      | Publicación de plantas . . . . .                                        | 14        |
| 8.3      | Sistema de reacciones . . . . .                                         | 14        |
| 8.4      | Comentarios . . . . .                                                   | 15        |
| <b>9</b> | <b>Problemas Encontrados y Soluciones</b>                               | <b>15</b> |

## **10 Conclusión**

**16**

## thesection Descripción General del Proyecto

**PlantApp** es una aplicación móvil y web de tipo red social temática, construida alrededor del universo de *Plants vs. Zombies*. Su objetivo principal es permitir a los usuarios registrar, compartir y explorar información detallada sobre plantas del juego: sus nombres, fotografías, descripciones, ataques, biografías, precios y tiempos de recarga.

La aplicación permite a cualquier usuario con cuenta publicar una “tarjeta de planta”, reaccionar a las publicaciones de otros usuarios con emojis al estilo de Facebook (like, love, haha, wow, sad, angry) y dejar comentarios de texto. Cada usuario puede gestionar únicamente sus propias publicaciones y comentarios, garantizando así integridad y control de contenido.

El backend está construido con **Spring Boot 4** y expone una API REST completa. La base de datos es **PostgreSQL** alojada en la nube mediante **Aiven**. El frontend fue desarrollado con **Flutter**, compilado tanto para web como para APK de Android. El backend se desplegó en **Render** mediante una imagen **Docker**.

### thesubsection Motivación

Plants vs. Zombies es uno de los juegos de estrategia más reconocidos a nivel global. Construir una red social temática alrededor de él permite practicar el desarrollo full-stack completo —desde la capa de datos hasta la interfaz móvil— en un contexto familiar y entretenido, al tiempo que se aplican patrones de diseño y principios de programación orientada a objetos de manera concreta y visible.

### thesubsection Estructura General del Proyecto

```
proyecto-tweeter/
+-- backend/                (Spring Boot + Maven)
|   +-- src/main/java/com/tweeter/backend/
|   |   +-- models/        (Entidades JPA)
|   |   +-- repository/    (Repositorios Spring Data)
|   |   +-- controllers/   (Controladores REST)
|   +-- Dockerfile
|   +-- pom.xml
+-- frontend/               (Flutter)
    +-- lib/
        +-- services/      (Capa de acceso a la API)
        +-- main.dart      (UI completa)
```

## thesection Tecnologías Utilizadas

| Tecnología            | Uso en el proyecto                             |
|-----------------------|------------------------------------------------|
| <b>Flutter (Dart)</b> | Interfaz de usuario: web y APK Android         |
| <b>Spring Boot 4</b>  | Framework backend, expone la API REST          |
| <b>PostgreSQL 17</b>  | Base de datos relacional en producción (Aiven) |

| Tecnología             | Uso en el proyecto                            |
|------------------------|-----------------------------------------------|
| <b>Hibernate / JPA</b> | Mapeo objeto-relacional de entidades          |
| <b>Docker</b>          | Contenedorización del backend para despliegue |
| <b>Render</b>          | Plataforma de despliegue en la nube           |
| <b>Aiven</b>           | Servicio administrado de PostgreSQL           |
| <b>Maven</b>           | Gestión de dependencias y build de Java       |

Table 1: Stack tecnológico del proyecto

## Principios de Diseño Utilizados

A lo largo del desarrollo de PlantApp se aplicaron de forma deliberada varios principios fundamentales de la ingeniería de software. Estos principios guiaron las decisiones de arquitectura y organización del código tanto en el backend como en el frontend.

### Principio de Responsabilidad Única (SRP)

El **Principio de Responsabilidad Única** (Single Responsibility Principle), parte de los principios SOLID, establece que una clase debe tener *una única razón para cambiar*. En otras palabras, cada clase debe encargarse de una sola responsabilidad dentro del sistema.

#### Aplicación en el Backend

En el backend, este principio se respetó al separar completamente las responsabilidades en capas distintas:

- **Modelos** (Tweet, Usuario, Reaccion, Comentario): solo representan la estructura de datos y su mapeo a la base de datos. No contienen lógica de negocio.
- **Repositorios** (TweetRepository, UsuarioRepository, etc.): solo se encargan del acceso a datos.
- **Controladores** (TweetController, AuthController, etc.): solo manejan las peticiones HTTP y delegan el trabajo a los repositorios.

Por ejemplo, `TweetController` no sabe cómo se guarda un dato en la base de datos; eso es responsabilidad de `TweetRepository`. Si mañana cambia el proveedor de base de datos, solo se modifica el repositorio, no el controlador.

```

1 @RestController
2 @RequestMapping("/tweets")
3 @CrossOrigin(origins = "*")
4 public class TweetController {
5
6     // Solo conoce al repositorio, no sabe nada de la DB
7     @Autowired
8     private TweetRepository repo;
9
10    @GetMapping
11    public List<Tweet> getAll() {
12        return repo.findAll(); // delega la responsabilidad

```

```
13     }
14
15     @PostMapping
16     public Tweet create(@RequestBody Tweet t) {
17         return repo.save(t);
18     }
19 }
```

Listing 1: Separación de responsabilidades en TweetController

### *thesubsubsection* Aplicación en el Frontend

En Flutter, la responsabilidad de comunicarse con la API se separó completamente de la UI. El archivo `tweet_service.dart` centraliza todas las llamadas HTTP, mientras que `main.dart` solo construye widgets y reacciona a eventos del usuario.

Si la URL del backend cambia (por ejemplo, de desarrollo a producción), solo se modifica `tweet_service.dart`, sin tocar ningún widget.

```
1 // tweet_service.dart - SOLO responsabilidad de HTTP
2 Future<List<dynamic>> getTweets() async {
3     final response = await http.get(Uri.parse('$baseUrl/tweets'));
4     if (response.statusCode == 200) return jsonDecode(response.body);
5     throw Exception('Error al cargar');
6 }
7
8 // main.dart - SOLO responsabilidad de UI
9 Future<void> _cargarTweets() async {
10     final data = await getTweets(); // delega al servicio
11     setState(() => tweets = data);
12 }
```

Listing 2: Separación entre servicio y UI en Flutter

### **thesubsubsection** Principio Abierto/Cerrado (OCP)

El **Principio Abierto/Cerrado** establece que las entidades de software deben estar *abiertas para extensión, pero cerradas para modificación*.

En este proyecto, el uso de `JpaRepository` como interfaz base para todos los repositorios es un ejemplo directo de este principio. Spring Data JPA provee una implementación completa sin necesidad de modificar ninguna clase existente. Cuando fue necesario agregar consultas personalizadas, bastó con declarar nuevos métodos en la interfaz sin tocar la implementación:

```
1 // Se extiende JpaRepository sin modificar nada interno
2 @Repository
3 public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
4     // Nueva funcionalidad agregada SIN modificar la clase base
5     Optional<Usuario> findByUsername(String username);
6 }
```

Listing 3: Extensión sin modificación en UsuarioRepository

Del mismo modo, cuando se agregaron las entidades `Reaccion` y `Comentario`, no fue necesario modificar `TweetController` ni `TweetRepository`; simplemente se crearon nuevos controladores y repositorios independientes.

### thesubsection Principio de Inversión de Dependencias (DIP)

El **Principio de Inversión de Dependencias** establece que los módulos de alto nivel no deben depender de los módulos de bajo nivel; ambos deben depender de abstracciones.

En Spring Boot, este principio se implementa automáticamente mediante la inyección de dependencias con `@Autowired`. Los controladores no instancian sus repositorios directamente (lo que generaría acoplamiento fuerte), sino que Spring los inyecta en tiempo de ejecución:

```
1 @RestController
2 public class TweetController {
3
4     // Spring inyecta la implementacion concreta en runtime
5     // El controlador solo conoce la ABSTRACCION (interfaz)
6     @Autowired
7     private TweetRepository repo; // interfaz, no clase concreta
8
9     @Autowired
10    private ReaccionRepository reaccionRepo;
11 }
```

Listing 4: Inversión de dependencias mediante `@Autowired`

Si en el futuro se cambia la implementación de `TweetRepository` (por ejemplo, para usar MongoDB en lugar de PostgreSQL), el controlador no requiere ningún cambio.

### thesubsection Principio DRY (Don't Repeat Yourself)

El principio **DRY** establece que *cada pieza de conocimiento debe tener una representación única, no ambigua y autoritativa dentro del sistema*.

En el frontend, la URL base del backend se definió una sola vez en `tweet_service.dart` como constante:

```
1 // Definida UNA sola vez
2 const String baseUrl = 'https://plantapp-api.onrender.com';
3
4 // Todos los metodos la reutilizan
5 Future<List<dynamic>> getTweets() async {
6     final response = await http.get(Uri.parse('$baseUrl/tweets'));
7     ...
8 }
9
10 Future<void> createTweet(Map<String, dynamic> data) async {
11     await http.post(Uri.parse('$baseUrl/tweets'), ...);
12 }
```

Listing 5: Constante central que evita repetición

Cuando fue necesario cambiar de `localhost:8080` a la URL de producción en Render, bastó con modificar *una sola línea* en todo el proyecto.

En el backend, el método constructor de widgets `_campo()` en el formulario de nueva planta evita repetir la declaración de cada `TextField`:

```

1 // SIN DRY habria que repetir este bloque 7 veces
2 Widget _campo(String label, TextEditingController ctrl,
3   {int maxLines = 1, TextInputType tipo = TextInputType.text}) {
4   return Padding(
5     padding: const EdgeInsets.only(bottom: 12),
6     child: TextField(
7       controller: ctrl,
8       maxLines: maxLines,
9       keyboardType: tipo,
10      decoration: InputDecoration(
11        labelText: label,
12        border: const OutlineInputBorder()
13      ),
14    ),
15  );
16 }
17
18 // USO: una sola linea por campo
19 _campo('Nombre', _nombreCtrl),
20 _campo('Descripcion', _descripCtrl, maxLines: 2),
21 _campo('Precio', _precioCtrl, tipo: TextInputType.number),

```

Listing 6: Método auxiliar que elimina repetición en formularios

### thesubsection Principio KISS (Keep It Simple, Stupid)

El principio **KISS** propone que los sistemas funcionan mejor cuando se mantienen simples. La simplicidad debe ser un objetivo clave en el diseño.

En este proyecto, la decisión de no implementar JWT ni Spring Security completo fue una aplicación consciente de KISS. Para el alcance del parcial, la autenticación se maneja con verificación directa de usuario y contraseña en el controlador, eliminando capas de complejidad innecesarias:

```

1 @PostMapping("/login")
2 public ResponseEntity<?> login(@RequestBody Usuario u) {
3   Optional<Usuario> encontrado =
4     repo.findByUsername(u.getUsername());
5
6   // Verificacion directa, sin tokens ni sesiones complejas
7   if (encontrado.isEmpty() ||
8     !encontrado.get().getPassword().equals(u.getPassword())) {
9     return ResponseEntity.status(401)
10       .body(Map.of("error", "Credenciales incorrectas"));
11   }
12
13   return ResponseEntity.ok(Map.of(
14     "mensaje", "Login exitoso",
15     "username", encontrado.get().getUsername()
16   ));
17 }

```

## Listing 7: Autenticación simple y directa (principio KISS)

**thesubsection Principio de Separación de Intereses (Separation of Concerns)**

Este principio establece que un programa debe dividirse en secciones distintas, donde cada una aborda una preocupación o aspecto separado del comportamiento del software.

En PlantApp, los intereses están claramente separados en capas:

| Capa                    | Responsabilidad                                  |
|-------------------------|--------------------------------------------------|
| <b>Modelos</b>          | Estructura de los datos y mapeo a tablas         |
| <b>Repositorios</b>     | Acceso y persistencia de datos                   |
| <b>Controladores</b>    | Manejo de peticiones HTTP y respuestas           |
| <b>Flutter Services</b> | Comunicación con la API REST                     |
| <b>Flutter Widgets</b>  | Presentación visual e interacción con el usuario |

Esta separación permite que, por ejemplo, el equipo de frontend trabaje en la UI sin necesidad de conocer cómo funciona internamente la base de datos, y viceversa.

**thesection Patrones de Diseño Utilizados****thesubsection Patrón Repository***thesubsubsection Definición*

El patrón **Repository** actúa como una colección de objetos en memoria. En lugar de escribir SQL directamente en la lógica de negocio, se crea una interfaz que define qué operaciones se pueden realizar sobre cada entidad. Spring Data JPA genera automáticamente la implementación a partir del nombre de los métodos.

*thesubsubsection Beneficio principal*

El código de negocio no sabe *cómo* se obtienen los datos, solo *qué* datos necesita. Esto desacopla completamente la lógica de negocio del mecanismo de persistencia.

*thesubsubsection Repositorios implementados*

```

1 @Repository
2 public interface TweetRepository extends JpaRepository<Tweet, Long> {
3     // Spring genera automaticamente todos los metodos CRUD
4     // findAll(), save(), deleteById(), findById()...
5 }

```

Listing 8: TweetRepository

```

1 @Repository
2 public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
3     // Spring genera: SELECT * FROM usuarios WHERE username = ?
4     Optional<Usuario> findByUsername(String username);
5 }

```

Listing 9: UsuarioRepository con consulta personalizada

```

1 @Repository
2 public interface ReaccionRepository extends JpaRepository<Reaccion,
   Long> {
3     List<Reaccion> findByTweetId(Long tweetId);
4     Optional<Reaccion> findByTweetIdAndAutor(Long tweetId, String
   autor);
5 }

```

Listing 10: ReaccionRepository con consultas derivadas

```

1 @Repository
2 public interface ComentarioRepository
3     extends JpaRepository<Comentario, Long> {
4     List<Comentario> findByTweetId(Long tweetId);
5 }

```

Listing 11: ComentarioRepository

## thesubsection Patrón MVC (Model-View-Controller)

### thesubsubsection Definición

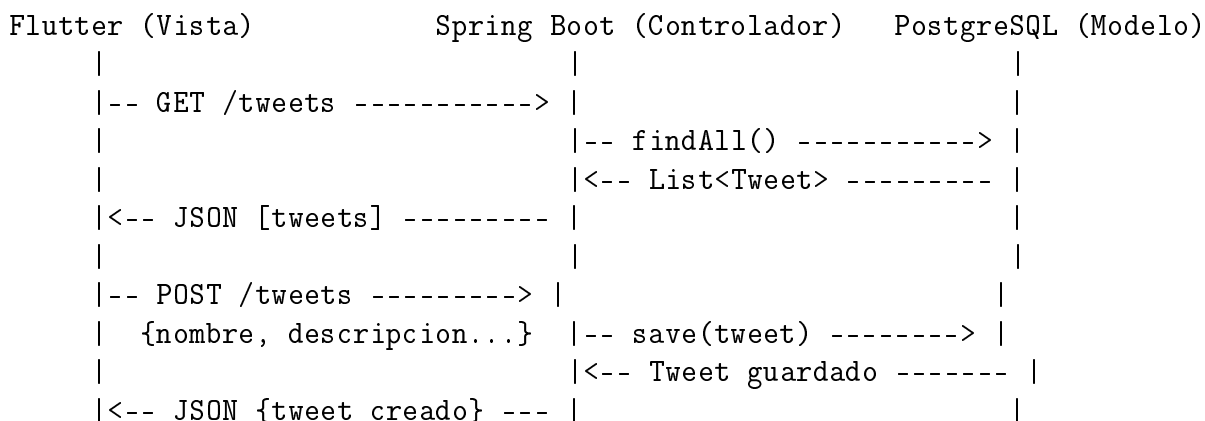
El patrón **MVC** divide la aplicación en tres capas: el **Modelo** maneja los datos, la **Vista** muestra la interfaz al usuario, y el **Controlador** recibe las peticiones y coordina entre los otros dos.

### thesubsubsection Aplicación en dos niveles

En este proyecto, MVC se aplica en dos niveles simultáneos:

- **Backend (Spring Boot):** las entidades JPA son el Modelo, los `@RestController` son los Controladores, y el JSON que retorna la API es la representación de la Vista.
- **Frontend (Flutter):** los widgets de Flutter son la Vista que consume la API y muestra los datos; `tweet_service.dart` actúa como el intermediario.

### thesubsubsection Diagrama de flujo MVC



### thesubsection Patrón de Diseño Facade (mencionado como extensión)

Aunque en este proyecto los controladores acceden directamente a los repositorios por simplicidad, el patrón **Facade** es una extensión natural que se podría aplicar. Este patrón proporciona una interfaz simplificada a un conjunto de interfaces en un subsistema.

La implementación de una clase `TweetFacade` centralizaría la coordinación entre `TweetRepository`, `ReaccionRepository` y `ComentarioRepository`, reduciendo el acoplamiento de los controladores:

```

1 @Service
2 public class TweetFacade {
3     private final TweetRepository tweetRepository;
4     private final ComentarioRepository comentarioRepository;
5     private final ReaccionRepository reaccionRepository;
6
7     // Constructor injection (DIP aplicado)
8     public TweetFacade(TweetRepository tweetRepo,
9                       ComentarioRepository comentRepo,
10                      ReaccionRepository reaccionRepo) {
11         this.tweetRepository = tweetRepo;
12         this.comentarioRepository = comentRepo;
13         this.reaccionRepository = reaccionRepo;
14     }
15
16     public Comentario agregarComentario(Long tweetId, String texto) {
17         Tweet tweet = tweetRepository.findById(tweetId)
18             .orElseThrow(() -> new RuntimeException(
19                 "Tweet no encontrado: " + tweetId));
20         Comentario c = new Comentario();
21         c.setTweetId(tweetId);
22         c.setContenido(texto);
23         return comentarioRepository.save(c);
24     }
25 }

```

Listing 12: Estructura de `TweetFacade` como extensión futura

## thesection Modelos / Entidades JPA

Los modelos son clases Java anotadas con `@Entity` que Hibernate mapea automáticamente a tablas de PostgreSQL. La anotación `spring.jpa.hibernate.ddl-auto=update` permite que las tablas se creen o actualicen automáticamente al iniciar la aplicación.

### thesubsection Tweet.java — Entidad principal

```

1 @Entity
2 @Table(name = "tweets")
3 public class Tweet {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private String nombre;           // Nombre de la planta

```

```

10 private String imageUrl;    // URL de la fotografia
11 private String descripcion; // Descripcion general
12 private String ataque;     // Tipo de ataque
13 private String biografia;  // Historia de la planta
14 private Double precio;     // Costo en soles de sol
15 private String recarga;    // Tiempo de recarga
16 private String autor;      // Usuario que la publico
17 private Integer reacciones = 0; // Contador legacy
18
19 // Getters y Setters...
20 }

```

Listing 13: Entidad Tweet con todos los campos de la planta

### thesubsection Usuario.java

```

1 @Entity
2 @Table(name = "usuarios")
3 public class Usuario {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(unique = true) // Username unico en la DB
10    private String username;
11
12    private String password;
13
14    // Getters y Setters...
15 }

```

Listing 14: Entidad Usuario

### thesubsection Reaccion.java

```

1 @Entity
2 @Table(name = "reacciones")
3 public class Reaccion {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private Long tweetId;
10    private String autor;
11    // Tipos posibles: LIKE, LOVE, HAHA, WOW, SAD, ANGRY
12    private String tipo;
13
14    // Getters y Setters...
15 }

```

Listing 15: Entidad Reaccion con tipo emoji

### thesubsection Comentario.java

```
1 @Entity
2 @Table(name = "comentarios")
3 public class Comentario {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     private Long tweetId;
10    private String autor;
11    private String contenido;
12
13    // Getters y Setters...
14 }
```

Listing 16: Entidad Comentario

thesection

Endpoints de la API REST

| Método           | Ruta                      | Descripción                      |
|------------------|---------------------------|----------------------------------|
| Autenticación    |                           |                                  |
| POST             | /auth/register            | Registrar nuevo usuario          |
| POST             | /auth/login               | Iniciar sesión                   |
| Plantas (Tweets) |                           |                                  |
| GET              | /tweets                   | Obtener todas las plantas        |
| POST             | /tweets                   | Crear nueva planta               |
| DELETE           | /tweets/{id}?autor=X      | Eliminar planta (solo autor)     |
| Reacciones       |                           |                                  |
| GET              | /reacciones/{tweetId}     | Obtener reacciones de una planta |
| POST             | /reacciones               | Agregar/cambiar/quitar reacción  |
| Comentarios      |                           |                                  |
| GET              | /comentarios/{tweetId}    | Listar comentarios de una planta |
| POST             | /comentarios              | Agregar comentario               |
| DELETE           | /comentarios/{id}?autor=X | Eliminar comentario (solo autor) |

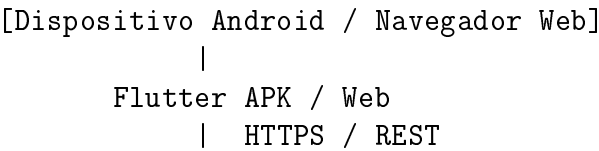
Table 2: Endpoints completos de la API REST

thesection

Arquitectura y Despliegue

thesubsection

Diagrama de arquitectura



```

      v
+-----+
| Render (plantapp-api.onrender.com) |
| +-----+ |
| | Docker Container | |
| | Spring Boot 4 + Maven | |
| | Puerto 8080 | |
| +-----+ |
+-----+
      | JDBC + SSL
      v
+-----+
| Aiven Cloud (PostgreSQL 17) |
| Tablas: tweets, usuarios, |
| reacciones, comentarios |
+-----+

```

### thesubsection Configuración de base de datos

La conexión a Aiven se configura mediante variables de entorno en Render, siguiendo el principio de no hardcodear credenciales en el código fuente:

Listing 17: application.properties con configuración de Aiven

```

spring.datasource.url=jdbc:postgresql://HOST:PORT/DATABASE?sslmode=
require
spring.datasource.username=avnadmin
spring.datasource.password=*****

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.
PostgreSQLDialect
server.port=8080

```

El parámetro `?sslmode=require` es obligatorio en Aiven para garantizar que la conexión sea cifrada.

### thesubsection Dockerfile

Listing 18: Dockerfile del backend

```

FROM openjdk:17-jdk-slim
WORKDIR /app
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]

```

El proceso de despliegue sigue estos pasos:

1. `mvn clean package -DskipTests` genera el archivo `.jar`
2. `docker build` construye la imagen con OpenJDK 17
3. `docker push andriuwu/plantapp-api` sube la imagen a Docker Hub

4. Render descarga la imagen y la ejecuta como servicio web

### thesubsection Compilación del APK

Para el frontend móvil, Flutter compila la misma base de código para múltiples plataformas:

Listing 19: Comandos de compilación Flutter

```
# Cambiar URL a produccion antes de compilar
# En tweet_service.dart:
# const String baseUrl = 'https://plantapp-api.onrender.com';

flutter build apk --release
# Genera: build/app/outputs/flutter-apk/app-release.apk

flutter build web --release
# Genera: build/web/
```

## thesection Funcionalidades de la Aplicación

### thesubsection Autenticación

Los usuarios pueden registrarse con un nombre de usuario y contraseña únicos. El sistema valida que el username no esté duplicado antes de crear la cuenta. El login verifica credenciales y redirige a la pantalla principal al éxito.

### thesubsection Publicación de plantas

El formulario de nueva planta incluye los siguientes campos en orden:

1. **Nombre** (obligatorio): nombre de la planta del juego
2. **Fotografía de planta**: URL de imagen pública
3. **Descripción**: resumen de la planta
4. **Ataque**: tipo y descripción del ataque
5. **Biografía**: historia de la planta en el universo del juego
6. **Precio**: costo en soles de sol (numérico)
7. **Recarga**: tiempo de recarga expresado en texto

### thesubsection Sistema de reacciones

Inspirado en las reacciones de Facebook, cada planta puede recibir seis tipos de reacción:

| Emoji           | Tipo  |
|-----------------|-------|
| (pulgar arriba) | LIKE  |
| (corazon)       | LOVE  |
| (risa)          | HAHA  |
| (sorpresa)      | WOW   |
| (triste)        | SAD   |
| (enojado)       | ANGRY |

Cada usuario puede tener solo una reacción activa por planta. Si reacciona dos veces con el mismo emoji, la reacción se elimina. Si reacciona con uno diferente, la reacción se actualiza. La lógica de toggle se implementa en el backend:

```

1 @PostMapping
2 public ResponseEntity<?> reaccionar(@RequestBody Reaccion r) {
3     Optional<Reaccion> existente =
4         repo.findByTweetIdAndAutor(r.getTweetId(), r.getAutor());
5
6     if (existente.isPresent()) {
7         Reaccion re = existente.get();
8         if (re.getTipo().equals(r.getTipo())) {
9             repo.delete(re); // misma reaccion -> quitar
10            return ResponseEntity.ok(Map.of("accion", "eliminada"));
11        } else {
12            re.setTipo(r.getTipo()); // diferente -> cambiar
13            repo.save(re);
14            return ResponseEntity.ok(Map.of("accion", "cambiada"));
15        }
16    }
17    repo.save(r); // nueva reaccion
18    return ResponseEntity.ok(Map.of("accion", "agregada"));
19 }

```

Listing 20: Lógica de toggle de reacciones

## thesubsection Comentarios

Cada publicación tiene una sección de comentarios desplegable. Cualquier usuario autenticado puede comentar en cualquier planta. Los usuarios solo pueden eliminar sus propios comentarios, validado tanto en el frontend (ocultando el botón) como en el backend (verificando el autor antes de borrar).

## thesection Problemas Encontrados y Soluciones

Durante el desarrollo se presentaron varios problemas técnicos relevantes desde el punto de vista de ingeniería de software:

| Problema                                                                    | Solución aplicada                                                                   |
|-----------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| El backend arrancaba y se cerraba inmediatamente                            | Faltaba la dependencia <code>spring-boot-starter-web</code> en <code>pom.xml</code> |
| Las dependencias estaban fuera del bloque <code>&lt;dependencies&gt;</code> | Corrección del XML malformado en <code>pom.xml</code>                               |
| Los archivos Java estaban en la raíz del proyecto                           | Movidos a <code>src/main/java/com/tweeter/backend/</code>                           |
| La imagen se recortaba en lugar de redimensionarse                          | Cambio de <code>BoxFit.cover</code> a <code>BoxFit.contain</code> en Flutter        |
| Tweets sin autor no podían eliminarse                                       | Eliminación temporal de la validación para limpiar datos huérfanos                  |

| Problema                                        | Solución aplicada                                                       |
|-------------------------------------------------|-------------------------------------------------------------------------|
| CORS bloqueaba las peticiones desde Flutter web | Anotación <code>@CrossOrigin(origins = “*”)</code> en los controladores |
| Conexión rechazada a Aiven                      | Faltaba el parámetro <code>?sslmode=require</code> en la URL JDBC       |

Table 3: Problemas encontrados durante el desarrollo

## thesection Conclusión

PlantApp demostró que es posible construir una aplicación full-stack funcional, desplegada en la nube y compilada como APK de Android, aplicando de forma consistente los principios fundamentales de diseño de software.

El **Principio de Responsabilidad Única** mantuvo cada clase enfocada en una sola tarea, lo que facilitó enormemente la depuración de errores: cuando el backend no arrancaba, era evidente que el problema estaba en la capa de configuración, no en los controladores ni en los modelos.

El principio **DRY** redujo la cantidad de código repetido, especialmente visible en el método `_campo()` de Flutter y en la constante `baseUrl`, que permitió cambiar de entorno de desarrollo a producción en Render con una sola modificación.

La **Separación de Intereses** entre el servicio HTTP de Flutter y los widgets de UI permitió que la interfaz evolucionara (agregando reacciones, comentarios, campos nuevos) sin necesidad de reescribir la lógica de comunicación con el backend.

El **Patrón Repository** de Spring Data JPA eliminó por completo la necesidad de escribir SQL, reduciendo el riesgo de errores y acelerando el desarrollo. El **Patrón MVC**, aplicado en dos niveles simultáneos (backend y frontend), garantizó que ambas capas pudieran desarrollarse y modificarse de forma independiente.

En conjunto, estos principios y patrones no son conceptos abstractos: fueron decisiones concretas que resolvieron problemas reales durante el desarrollo de esta práctica.

*Andrés Marcelino Escobar*  
Principios de Diseño de Software — Mayo 2026